

AI DISCUSSION AND TESTING NOTES

Course: Data Structures and Algorithms

Assignment 1

PART A - Indicative Questions and Answers

1) Why do I get the error "Could not find or load main class"?

Answer:

This error usually appears when Java cannot locate the compiled class file.

The most common reasons are:

- wrong classpath
- wrong package name
- the class was not compiled in the expected location

2) What does -Xint do and why do we use it?

Answer:

The -Xint option forces the JVM to run in interpreted mode only, without JIT optimizations.

It is used here because the assignment asks for cleaner and more stable time measurements in nanoseconds.

3) Why is binary search faster than BST in search?

Answer:

Binary search works on a sorted array and always splits the search interval in half.

This gives logarithmic behavior with good memory locality.

A BST also has average $O(\log N)$ search, but its shape depends on the inserted keys.

4) Why is insert in a sorted array $O(N)$?

Answer:

Even if the insertion position is found quickly, the elements to the right of that position must be shifted.

That shifting is linear in the number of elements.

5) Why do we still use sorted arrays if insert is expensive?

Answer:

Because sorted arrays provide:

- very fast search
- simple implementation
- good cache locality

They are useful when searches are frequent and updates are rare.

6) What are the best, average and worst case complexities in a BST?

Answer:

For search, insert and delete:

- best case: $O(1)$ if the key is found immediately at the root
- average case: $O(\log N)$
- worst case: $O(N)$ if the tree becomes skewed

7) What is the base of the logarithm?

Answer:

In binary search and BST analysis, the logarithm is base 2.

In asymptotic complexity notation, the base is not important.

8) What are inner classes?

Answer:

Inner classes are classes defined inside another class.

They are useful when helper structures belong logically to one outer class.

9) Can two custom classes be used to store experiment results?

Answer:

Yes.

One class can store the 4 operation results for one value of N .

Another class can store the measurements for one operation:

time, operations and levels for the 3 structures.

10) Can a helper class be used to group together the three data structures of the experiment?

Answer:

Yes.

This is good design because it groups together the Dynamic BST, the Array BST and the Sorted Array structure used in one experiment run.

11) Does K in the experiment mean the number of repetitions?

Answer:

Yes.

K is the number of repetitions of each operation for a given N , so that average measurements can be computed.

12) Can the capacity of the array-based structures depend on N and K instead of being fixed?

Answer:

Yes.

A reasonable choice is capacity = $N + K$ (+ a small safety margin), because the structures initially store N keys and in operation A up to K new insertions may occur.

13) In the ArrayBST, can I use int instead of the IELEM abstraction from the tutorial?

Answer:

Yes.

The assignment explicitly states that the keys are positive integers.

So int is sufficient and simpler.

14) In the array-based tree, should I use null or -1?

Answer:

-1 should be used.

In the array implementation, children are represented by integer indices, not object references.

So -1 plays the role of null.

15) Where is the instruction “replace pointer to node by row index of the array” implemented?

Answer:

It is implemented in the ArrayBST by replacing:

- node.left with table[LEFT][index]
- node.right with table[RIGHT][index]
- root pointer with an integer root index
- null with -1

16) Does the free-list of available rows behave like a stack?

Answer:

Yes.

The assignment explicitly describes insertion and removal from the top of the free list.

So freeNode corresponds to push and getNode corresponds to pop.

17) Would a queue be appropriate instead of a stack for the free list?

Answer:

No.

The specification clearly describes LIFO behavior (top insertion and top removal), which corresponds to a stack, not a queue.

18) What is gained by using a static 2D array in the ArrayBST?

Answer:

It avoids dynamic allocation for every node, gives more control over memory usage, and may improve locality of reference.

19) Is it necessary to use a 3 x K array instead of K x 3 in Java?

Answer:

Yes, it is a good optimization.

A K x 3 array in Java creates K small arrays, while a 3 x K array creates only 3 internal arrays.

The assignment itself mentions this optimization.

20) Can the dynamic-memory BST be based on the tutorial code?

Answer:

Yes.

The assignment allows the use of an existing BST implementation as a starting point. It just has to be adapted to the assignment requirements.

21) What do the tables printed by the experiment show?

Answer:

For each N and for each operation, they show:

- average number of operations
- average execution time in nanoseconds
- average number of levels

for the 3 different structures.

22) What does the number of levels mean?

Answer:

For the trees, it is the number of nodes visited during the operation.

For binary search in the sorted array, it is the number of array cells/probes examined.

23) Why do Dynamic BST and Array BST have almost identical operations and levels?

Answer:

Because they follow the same BST logic.

They differ mainly in memory representation, not in the search-tree algorithm itself.

24) Why does the sorted array perform much worse in insert and delete?

Answer:

Because elements must be shifted in order to preserve sorted order.

25) Why is range search $O(K + \log N)$?

Answer:

Because first the range must be located, which takes logarithmic work, and then K result elements must be reported.

26) Why does inorder traversal cost $O(N)$?

Answer:

Because every node of the tree must be visited exactly once.

27) Is inorder relevant to the sorted array structure?

Answer:

Not as a tree traversal.

The array is already sorted, so inorder is not needed there.

28) Why were helper classes for storing results considered?

Answer:

Because the experiment produces structured data:

for each N , for each operation, and for each structure.

A dedicated design helps organization and readability.

29) Why are comments in English acceptable/useful?

Answer:

Because they make the code easier to read and explain the implementation clearly, which is one of the assignment requirements.

30) Why are duplicate inserts ignored?

Answer:

Because the assignment explicitly states that if the key already exists, it must not be inserted again.

The keys in the structures are unique.

31) Why are the random initial numbers required to be non-sorted and unique?

Answer:

Because the assignment wants random unique initial keys and not already sorted input.

This helps produce a more representative average-case BST shape.

32) Why are the structures reused across operations A, B, C and D for each N ?

Answer:

Because the assignment clarifies that later operations may be performed on structures already modified by earlier ones.

33) Why is `System.nanoTime()` used?

Answer:

Because the assignment explicitly asks for execution time in nanoseconds.

34) Why are averages used instead of one single measurement?

Answer:

Because one measurement can be noisy or unrepresentative.

Multiple repetitions give a more stable estimate.

35) Why can time fluctuate even if complexity is theoretically clear?

Answer:

Because runtime depends not only on asymptotic complexity, but also on system-level effects such as memory access, JVM behavior and small timing noise.

36) Why can the sorted array be faster in search but worse in insert/delete?

Answer:

Because binary search is very efficient on a sorted contiguous array, while updates require shifts.

37) Why are the dynamic BST and array BST expected to have the same asymptotic complexity?

Answer:

Because both implement the same BST logic.

The difference lies in how nodes are stored.

38) Why can the array BST still differ in absolute time from the dynamic BST?

Answer:

Because memory layout and cache locality differ, even if the number of logical steps is the same.

39) What does the free list save us from?

Answer:

It allows reuse of deleted rows and avoids searching the whole table for an empty position.

40) Why is the free list stored in the RIGHT field for unused rows?

Answer:

Because the assignment explicitly gives the third column a dual role: right child for used nodes and next free row for unused ones.

PART B - Test Programs Used

The following standalone test programs were created in order to validate the correctness of the implementations before running the experimental measurements.

The tests cover:

- insertion
- duplicate insertion
- search for existing and non-existing keys
- range search
- deletion of leaf / one-child / two-child nodes
- inorder correctness
- internal free-list correctness in the ArrayBST
- capacity limits in the sorted array structure

B1 - Test for DynamicMemoryBST

File:

MainTestDynamicBST.java

Purpose:

- check insertion order
- verify duplicate insertion is ignored
- verify inorder traversal is sorted
- test search
- test range search
- test delete on several cases

Code used:

```
import java.util.List;
```

```
public class MainTestDynamicBST {

    public static void main(String[] args) {
        DynamicMemoryBST tree = new DynamicMemoryBST();

        System.out.println("=====");
        System.out.println("TEST DynamicMemoryBST");
        System.out.println("=====");

        tree.printName();

        int[] keys = {50, 30, 70, 20, 40, 60, 80};

        System.out.println("\nInserting keys:");
        for (int key : keys) {
            tree.resetMetrics();
            tree.insert(key);
            System.out.println("Inserted " + key +
                " | operations = " + tree.getOperations() +
                " | levels = " + tree.getLevels());
        }
    }
}
```

```
tree.resetMetrics();
tree.insert(30);
System.out.println("\nTried inserting duplicate key 30" +
    " | operations = " + tree.getOperations() +
    " | levels = " + tree.getLevels());

System.out.println("\nInorder traversal (should be sorted):");
tree.inorder();

System.out.println("\nSearch tests:");
```

```
tree.resetMetrics();
int result1 = tree.search(40);
System.out.println("Search 40 -> " + result1 +
    " | operations = " + tree.getOperations() +
    " | levels = " + tree.getLevels());
```

```
tree.resetMetrics();
int result2 = tree.search(100);
System.out.println("Search 100 -> " + result2 +
    " | operations = " + tree.getOperations() +
    " | levels = " + tree.getLevels());
```

```
System.out.println("\nRange search [25, 65]:");
tree.resetMetrics();
List<Integer> range = tree.rangeSearch(25, 65);
System.out.println("Result = " + range +
    " | operations = " + tree.getOperations() +
    " | levels = " + tree.getLevels());
```

```
System.out.println("\nDelete tests:");
```

```
tree.resetMetrics();
boolean deleted1 = tree.delete(20);
System.out.println("Delete 20 -> " + deleted1 +
    " | operations = " + tree.getOperations() +
    " | levels = " + tree.getLevels());
tree.inorder();
```

```
tree.resetMetrics();
boolean deleted2 = tree.delete(30);
System.out.println("Delete 30 -> " + deleted2 +
    " | operations = " + tree.getOperations() +
    " | levels = " + tree.getLevels());
tree.inorder();
```

```
tree.resetMetrics();
```



```

        boolean deleted3 = tree.delete(50);
        System.out.println("Delete 50 -> " + deleted3 +
            " | operations = " + tree.getOperations() +
            " | levels = " + tree.getLevels());
        tree.inorder();

        tree.resetMetrics();
        boolean deleted4 = tree.delete(999);
        System.out.println("Delete 999 -> " + deleted4 +
            " | operations = " + tree.getOperations() +
            " | levels = " + tree.getLevels());

        System.out.println("\nFinal inorder:");
        tree.inorder();

        System.out.println("\n=====");
        System.out.println("END OF TEST");
        System.out.println("=====");
    }
}

```

B2 - Test for ArrayBST

File:

MainTestArrayBST.java

Purpose:

- validate the BST logic in the array implementation
- verify free-list initialization
- verify node reuse after deletions
- verify duplicate insertion is ignored
- verify search, range search, inorder and delete

Code used:

```

import java.util.List;

public class MainTestArrayBST {

    public static void main(String[] args) {

        System.out.println("=====");
        System.out.println("TEST ArrayBST");
        System.out.println("=====");

        ArrayBST tree = new ArrayBST(20);
    }
}

```

```

tree.printName();

System.out.println("\nInitial free list:");
tree.printTable();

int[] keys = {50, 30, 70, 20, 40, 60, 80};

System.out.println("\nInserting keys:");
for (int key : keys) {
    tree.resetMetrics();
    tree.insert(key);
    System.out.println("Inserted " + key +
        " | operations = " + tree.getOperations() +
        " | levels = " + tree.getLevels());
}

System.out.println("\nTable after inserts:");
tree.printTable();

tree.resetMetrics();
tree.insert(30);
System.out.println("\nTried inserting duplicate 30");
System.out.println("operations = " + tree.getOperations() +
    " | levels = " + tree.getLevels());

System.out.println("\nInorder traversal (must be sorted):");
tree.inorder();

System.out.println("\nSearch tests:");

tree.resetMetrics();
System.out.println("Search 40 -> " + tree.search(40));
printMetrics(tree);

tree.resetMetrics();
System.out.println("Search 999 -> " + tree.search(999));
printMetrics(tree);

System.out.println("\nRange [25, 65]:");

tree.resetMetrics();
List<Integer> range = tree.rangeSearch(25, 65);
System.out.println("Result = " + range);
printMetrics(tree);

System.out.println("\nDelete leaf (20):");
tree.resetMetrics();
System.out.println("Deleted 20 -> " + tree.delete(20));

```

```

printMetrics(tree);
tree.inorder();
tree.printTable();

System.out.println("\nDelete node with 1 child (30 or depending state):");
tree.resetMetrics();
System.out.println("Deleted 30 -> " + tree.delete(30));
printMetrics(tree);
tree.inorder();
tree.printTable();

System.out.println("\nDelete node with 2 children (50 root):");
tree.resetMetrics();
System.out.println("Deleted 50 -> " + tree.delete(50));
printMetrics(tree);
tree.inorder();
tree.printTable();

System.out.println("\nDelete non-existing (999):");
tree.resetMetrics();
System.out.println("Deleted 999 -> " + tree.delete(999));
printMetrics(tree);

System.out.println("\nFinal inorder:");
tree.inorder();

System.out.println("\nFinal table:");
tree.printTable();

System.out.println("\n=====");
System.out.println("END TEST");
System.out.println("=====");
}

private static void printMetrics(ArrayBST tree) {
    System.out.println("operations = " + tree.getOperations() +
        ", levels = " + tree.getLevels());
}
}

```

B3 - Test for SortedArrayBinarySearch

File:

MainTestSortedArrayBinarySearch.java

Purpose:

- verify sorted insertion

- verify duplicate insertion is ignored
- verify binary search
- verify range search
- verify deletion and shifting
- verify capacity overflow behavior

Code used:

```
import java.util.List;

public class MainTestSortedArrayBinarySearch {

    public static void main(String[] args) {
        System.out.println("=====");
        System.out.println("TEST SortedArrayBinarySearch");
        System.out.println("=====");

        SortedArrayBinarySearch structure = new SortedArrayBinarySearch(10);

        structure.printName();

        System.out.println("\nInitial array:");
        structure.printArray();
        System.out.println("size = " + structure.size());

        int[] keys = {50, 30, 70, 20, 40, 60, 80};

        System.out.println("\nInserting keys:");
        for (int key : keys) {
            structure.resetMetrics();
            structure.insert(key);
            System.out.print("Inserted " + key + " -> ");
            structure.printArray();
            printMetrics(structure);
        }

        structure.resetMetrics();
        structure.insert(30);
        System.out.println("\nTried inserting duplicate 30:");
        structure.printArray();
        printMetrics(structure);

        System.out.println("\nSearch tests:");

        structure.resetMetrics();
        int result1 = structure.search(40);
        System.out.println("Search 40 -> " + result1);
        printMetrics(structure);
    }
}
```

```

structure.resetMetrics();
int result2 = structure.search(999);
System.out.println("Search 999 -> " + result2);
printMetrics(structure);

System.out.println("\nRange search [25, 65]:");
structure.resetMetrics();
List<Integer> range = structure.rangeSearch(25, 65);
System.out.println("Result = " + range);
printMetrics(structure);

System.out.println("\nDelete tests:");

structure.resetMetrics();
boolean deleted1 = structure.delete(20);
System.out.println("Delete 20 -> " + deleted1);
structure.printArray();
printMetrics(structure);

structure.resetMetrics();
boolean deleted2 = structure.delete(50);
System.out.println("Delete 50 -> " + deleted2);
structure.printArray();
printMetrics(structure);

structure.resetMetrics();
boolean deleted3 = structure.delete(999);
System.out.println("Delete 999 -> " + deleted3);
structure.printArray();
printMetrics(structure);

System.out.println("\nFill to capacity:");
try {
    structure.resetMetrics();
    structure.insert(10);
    structure.insert(90);
    structure.insert(100);
    structure.printArray();
    printMetrics(structure);

    System.out.println("\nTrying to insert into full array:");
    structure.resetMetrics();
    structure.insert(110);
} catch (IllegalStateException e) {
    System.out.println("Caught expected exception: " + e.getMessage());
}

```

```

        System.out.println("\nFinal array:");
        structure.printArray();
        System.out.println("Final size = " + structure.size());

        System.out.println("\n=====");
        System.out.println("END TEST");
        System.out.println("=====");
    }

    private static void printMetrics(SortedArrayBinarySearch structure) {
        System.out.println("operations = " + structure.getOperations()
            + ", levels = " + structure.getLevels());
    }
}

```

PART C - Summary of What the Tests Validate

DynamicMemoryBST:

- correctness of BST insertion
- rejection of duplicates
- sorted inorder traversal
- search correctness
- range search correctness
- deletion correctness

ArrayBST:

- correctness of BST insertion using indices instead of pointers
- correct initialization of the free-list
- correct reuse of freed rows
- duplicate handling
- search / range / delete correctness
- internal table consistency

SortedArrayBinarySearch:

- array remains sorted after insertions
- duplicates are ignored
- binary search works correctly
- range search returns correct interval
- deletions shift elements correctly
- overflow case is handled correctly

PART D - Note on the Role of AI Assistance

Discussions with an AI tool were used in a supportive way for:

- concept clarification
- interpretation of the assignment requirements

- debugging of execution and package/classpath issues
- explanation of asymptotic behavior and experimental design

The implementation, the experimental setup, the execution of the tests and the interpretation of the final results were carried out by the authors.

+The theoretical complexity results presented in this section are based on the lecture material "Trees" and "Searching".